

```
>>> counter = Counter({'pen': 8,
'pencil': 8, 'book': 4})
```

Update with positive values

```
>>> counter.update({'pen':2, 'pencil': 1, 'book': 7})
```

```
>>> counter
```

```
Counter({'book': 11, 'pen': 10, 'pencil': 9})
```

Update with negative values

```
>>> counter.update({'pen': -2})
```

```
>>> counter
```

```
Counter({'book': 4, 'pen': 6, 'pencil': 8})
```

- In order to fetch a count corresponding to a key, we can query the Counter like a standard python Dictionary.

In the aforementioned example, if we wish to fetch the count corresponding to the key 'pen'.

```
>>> counter['pen']
```

```
8
```

If the key is not present in the Counter, it will return a 0 value. Unlike a Dictionary, which throws a `KeyError` in case the key is not found.

- If we wish to iterate over all the elements of a Counter, we can use the `elements()` method, which returns an iterator of elements with each element repeated as many times as its count.

```
>>> counter = Counter({'a':3, 'b':2, 'c':3, 'd':0,
'e':-2})
```

```
>>> list(counter.elements())
```

```
['a', 'a', 'a', 'b', 'b', 'c', 'c', 'c']
```

The above example illustrates the usage of the `elements()` method. Evidently, any keys that have a value of zero or lesser are ignored by the method.

If a key has an associated value as a float data type, you end up getting a `TypeError`

```
>>> counter = Counter({'a': 1.112, 'b': 2, 'c': 3})
```

```
>>> list(counter.elements())
```

```
TypeError: integer argument expected, got float
```

But such an operation is against the nature of Counter since by definition it clearly states it is meant for counting hashable objects. And a count of an object will always be a positive integer, hence negatives and floats are precluded.